

HDOC DAY – 17

QUESTION:

Write a menu-driven C program using **switch-case** to perform digit-based operations on a positive integer. Define **one separate user-defined function for each menu option**, with *void return type* and with a *simple function call (without call by value/reference)*. Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- Generate the largest possible number using its digits
- Generate the smallest possible number using its digits.
- Swap its first and last digits.
- Check whether it is a palindrome number.

Algorithm:

1. Start.
2. Display the menu:
 - 1 → Generate largest possible number using its digits.
 - 2 → Generate smallest possible number using its digits.
 - 3 → Swap first and last digits.
 - 4 → Check whether the number is palindrome.
3. Ask the user to enter a choice.
4. Use switch-case to perform the selected operation.
5. Call the corresponding user-defined function.
6. If the choice is invalid, display "Invalid choice".
7. Stop.

Function 1: Largest Possible Number

1. Read a positive integer.
2. Extract each digit using % 10.
3. Count how many times each digit from 0 to 9 occurs.

4. Arrange the digits from 9 down to 0.
5. Display the resulting largest number.

Function 2: Smallest Possible Number

1. Read a positive integer.
2. Extract and count all digits.
3. Find the smallest **non-zero** digit and print it first.
4. Print all zeroes next.
5. Print the remaining digits in increasing order.
6. This prevents the resulting number from beginning with 0.
7. Display the smallest possible number.

Function 3: Swap First and Last Digits

1. Read a positive integer.
2. Find the last digit using % 10.
3. Find the place value of the first digit.
4. Find the first digit.
5. Remove the first and last digits from their original positions.
6. Put the last digit in the first position.
7. Put the first digit in the last position.
8. Display the resulting number.

Function 4: Palindrome Number

1. Read a positive integer.
2. Store the original number.
3. Reverse the number digit by digit.
4. Compare the reversed number with the original number.
5. If both are equal, display "Palindrome number".
6. Otherwise, display "Not a palindrome number".

C program:

```
#include <stdio.h>

/* Function declarations */
void largestNumber(void);
void smallestNumber(void);
void swapFirstLast(void);
void checkPalindrome(void);

int main()
{
    int choice;

    printf("----- MENU ----- \n");
    printf("1. Generate largest possible number\n");
    printf("2. Generate smallest possible number\n");
    printf("3. Swap first and last digits\n");
    printf("4. Check palindrome number\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            largestNumber();
            break;

        case 2:
```

```

        smallestNumber();

        break;

    case 3:

        swapFirstLast();

        break;

    case 4:

        checkPalindrome();

        break;

    default:

        printf("Invalid choice\n");

}

return 0;

}

/* Function to generate the largest possible number */
void largestNumber(void)
{
    int num, digit, i;

    int count[10] = {0};

    printf("Enter a positive integer: ");

    scanf("%d", &num);

    while (num > 0)

```

```

{
    digit = num % 10;
    count[digit]++;
    num = num / 10;
}

printf("Largest number = ");

for (i = 9; i >= 0; i--)
{
    while (count[i] > 0)
    {
        printf("%d", i);
        count[i]--;
    }
}

printf("\n");
}

/* Function to generate the smallest possible number */
void smallestNumber(void)
{
    int num, digit, i;
    int count[10] = {0};

    printf("Enter a positive integer: ");
    scanf("%d", &num);

```

```
while (num > 0)
{
    digit = num % 10;
    count[digit]++;
    num = num / 10;
}
```

```
printf("Smallest number = ");
```

```
/* Print the smallest non-zero digit first */
```

```
for (i = 1; i <= 9; i++)
```

```
{
    if (count[i] > 0)
    {
        printf("%d", i);
        count[i]--;
        break;
    }
}
```

```
/* Print remaining digits in ascending order */
```

```
for (i = 0; i <= 9; i++)
```

```
{
    while (count[i] > 0)
    {
        printf("%d", i);
        count[i]--;
    }
}
```

```

    }
}

printf("\n");
}

/* Function to swap the first and last digits */
void swapFirstLast(void)
{
    int num, first, last, divisor = 1;
    int middle, result;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    last = num % 10;

    while (num / divisor >= 10)
    {
        divisor = divisor * 10;
    }

    first = num / divisor;

    /* Remove first and last digits */
    middle = (num % divisor) / 10;

    /* Form the new number */

```

```

    result = last * divisor + middle * 10 + first;

    printf("After swapping = %d\n", result);
}

/* Function to check whether a number is palindrome */
void checkPalindrome(void)
{
    int num, original, reverse = 0, digit;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    original = num;

    while (num > 0)
    {
        digit = num % 10;
        reverse = reverse * 10 + digit;
        num = num / 10;
    }

    if (original == reverse)
    {
        printf("Palindrome number\n");
    }
    else
    {

```

```

printf("Not a palindrome number\n");

}

}

```

Compilation and execution:

```

(kali@kali)~$ vi h17.c
(kali@kali)~$ gcc h17.c -o h17
(kali@kali)~$ ./h17
  MENU  ———
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome number
Enter your choice: █

```

Test case table:

Test case	choice	Input	Expected outcome	purpose
1	1	31042	Largest number = 43210	Normal case
2	1	55231	Largest number = 55321	Repeated digits
3	2	31042	Smallest number = 10234	Number containing zero
4	2	95321	Smallest number = 12359	Number without zero
5	3	12345	After swapping = 52341	Normal swap
6	4	12321	Palindrome number	Palindrome case
7	5	--	Invalid choice	Invalid menu option

```

$ ./h17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome number
Enter your choice: 31042
Invalid choice

(kali㉿kali)-[~]
$ ./h17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome number
Enter your choice: 1
Enter a positive integer: 31042
Largest number = 43210

(kali㉿kali)-[~]
$ ./h17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome number
Enter your choice: 1
Enter a positive integer: 55231
Largest number = 55321

(kali㉿kali)-[~]
$ ./h17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome number
Enter your choice: 2

```